ECS 2025

Power Up Your M365 Development Game:

Embracing React Functional Components and Hooks

DON KIRKHAM

Microsoft MVP, M365 Development

Premium Partner

Microsoft

Premium Sponsor

EXPERTS INSIDE
in partnership with EasyLife 365

Technology Partner

run events

Diamond Sponsor

AURUM
docusign
dox42
LightningTools
ShareGate

Platinum Sponsor

AvePoint
Capgemini
CoreView
empowerID
glueckkanja
involv
BUILT ON MICROSOFT 365
ITENOS
IT's us.
Jabra GN
nintex
Nordcloud
NUTANIX
q.beyond
Rencore
Syskit
Xylos

Gold Sponsor

365TRIBE
ADOPTION. CHAMPIONED
BCC
BlueCallom
BARCO ClickShare
DEKOM | neat.
dstny
d.velop
Gamma
loopup
protiviti
Global Business Consulting
Red Hat
skybow
SquaredUp
U2U
UDS
VISIONET
Engineering. Simplified.

Silver Sponsor

Apvee
do more with less
brandplane
CROSSWARE
CYQUEO
Cyber-Security Solutions
daenet
datacamp
EPOS
THE POWER OF AUDIO
ichicraft
Lucid
OnTime
IF TIME MATTERS...
opium.
panagenda
rocketta
Pearson | VUE
pointfire
Multilingual SharePoint
PowerSyncPro
Integrate. Collaborate. Migrate.
Yealink

Bronze Sponsor

Alpha Variance Solutions | GmbH
Analytics365
Callroute
epiq
HEINRICH & REUTER SOLUTIONS GMBH
HeiReS
DESIGN | DEVELOPMENT | TRAINING
Intelligent Decisioning
iThink365
kurmi
leapwork
sapio365
ITsources
SOLUTIONS 2SHARE
WristPlanner

Startup Silver

Graia
ShArc

Startup Bronze

AXL hub_
MEGALADATA
TeamsFox
vshosting

Startup Sponsor

emgc

@DONKIRKHAM

MICROSOFT MVP,
M365 DEVELOPMENT

encora Enterprise Architect

https://donkirkham.com

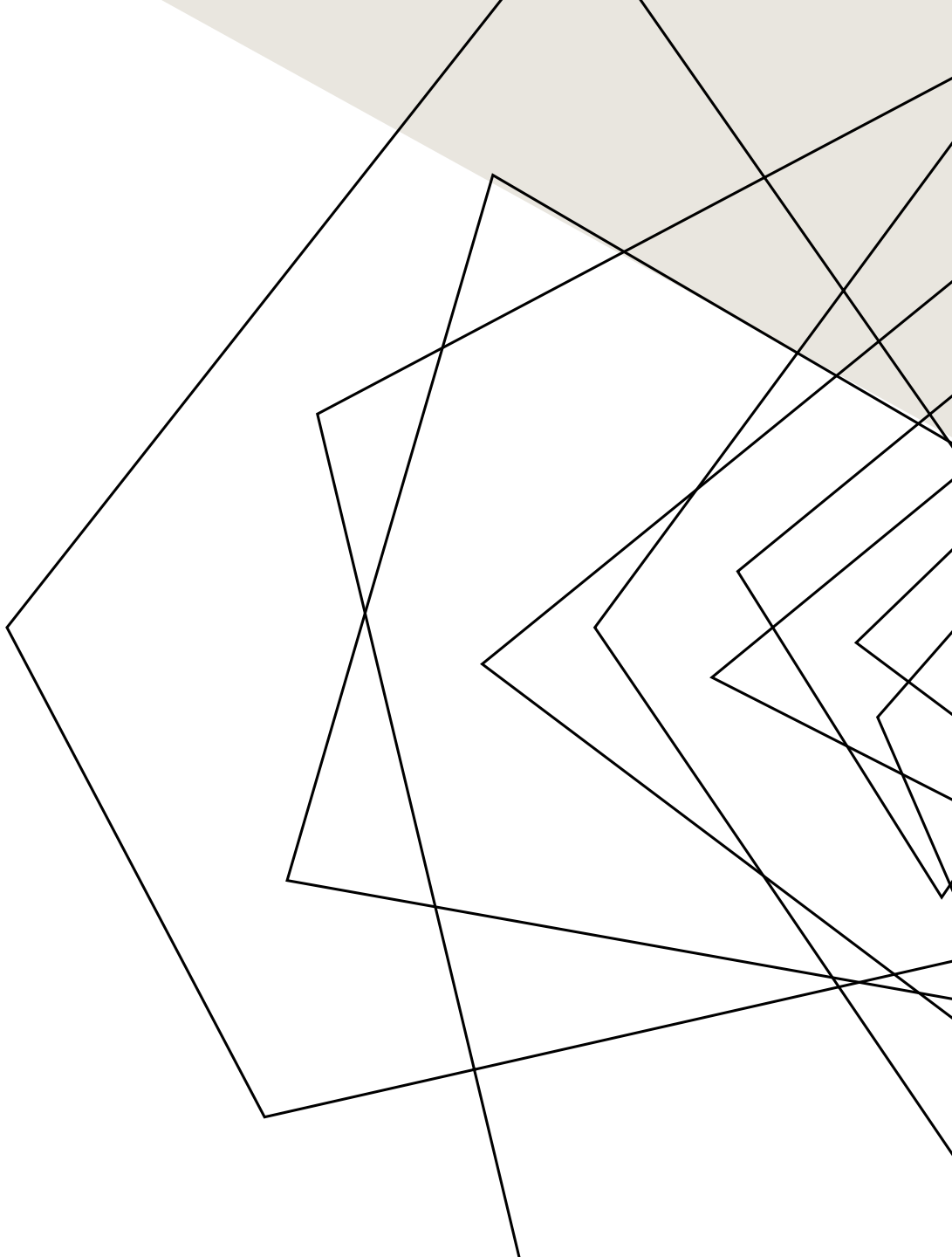
@DonKirkham

/in/DonKirkham

DonKirkham

AGENDA

- What are Function(al) Components and Hooks
- Classes vs Function Components
- Converting your SPFx project
- Hooks, Hooks, Hooks!



FUNCTIONAL COMPONENTS + HOOKS (FC+HOOKS)

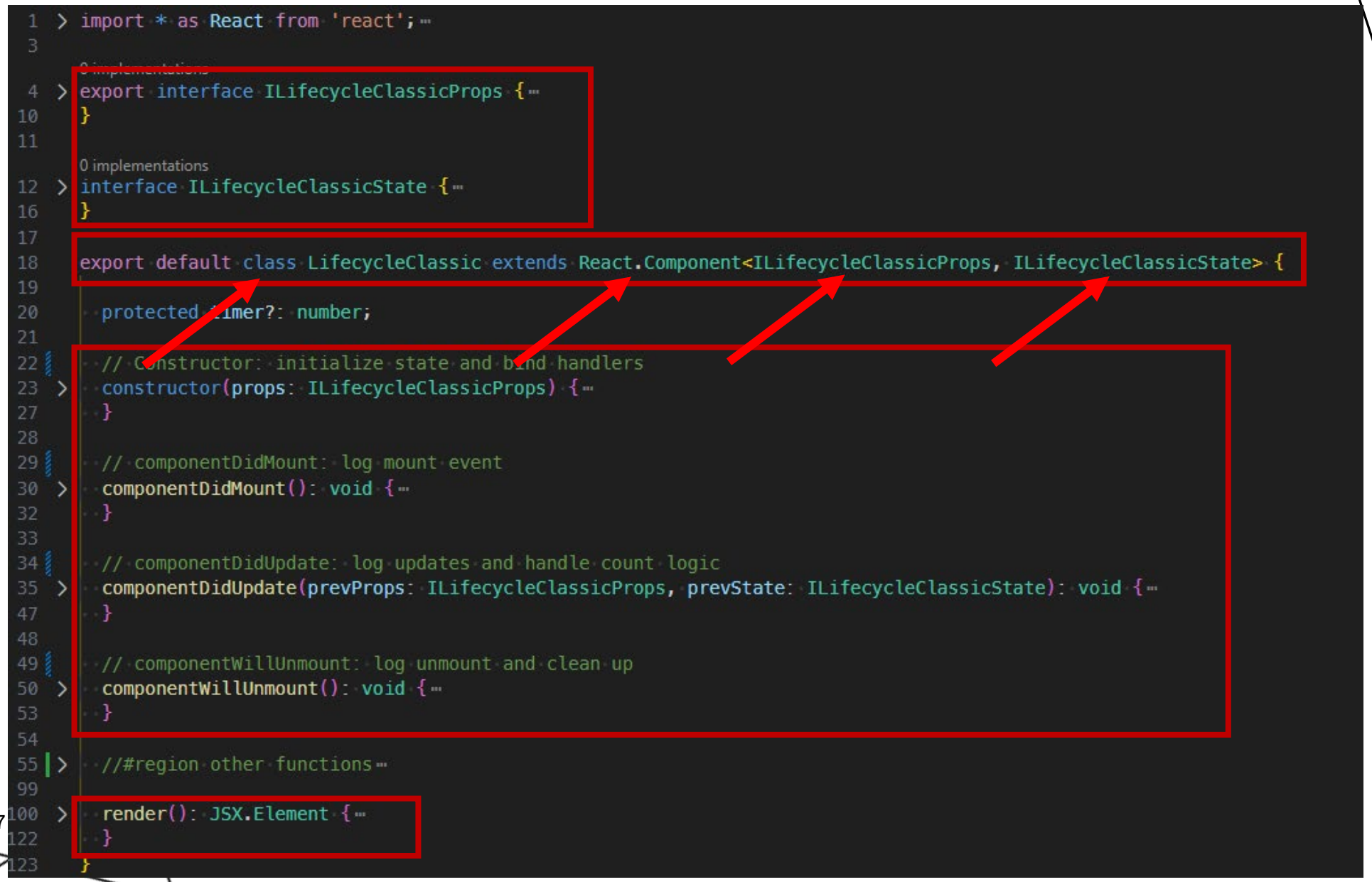
THE HISTORY

- 2015 – FCs introduced as Class alternative
  - Stateless
- 2019 – React Hooks introduced in React 16.8
  - Allowed granular state management in FC

- SharePoint Framework (SPFx)
  - Microsoft Yeoman Generator
    - Default scaffolds with Class Components
    - FC+Hooks only available in SPFx 1.9+

- Function vs Functional
  - Functional confusing based on Functional Computing
  - Functional Component is a defined React element
  - Function results in cleaner code
    - Functional uses implicit properties (inconsistent)
  - “Preferred” way to do function-based components

CLASS COMPONENT (DEFAULT SPFX)



```
import * as React from 'react';…

export interface ILifecycleClassicProps {…
}

0 implementations
interface ILifecycleClassicState {…
}

export default class LifecycleClassic extends React.Component<ILifecycleClassicProps, ILifecycleClassicState> {

  protected timer?: number;

  // Constructor: initialize state and bind handlers
  constructor(props: ILifecycleClassicProps) {…
  }

  // componentDidMount: log mount event
  componentDidMount(): void {…
  }

  // componentDidUpdate: log updates and handle count logic
  componentDidUpdate(prevProps: ILifecycleClassicProps, prevState: ILifecycleClassicState): void {…
  }

  // componentWillUnmount: log unmount and clean up
  componentWillUnmount(): void {…
  }

  //#region other functions…

  render(): JSX.Element {…
  }
}
```

WHAT ARE FUNCTION COMPONENTS?

- Simply, JavaScript Functions
  - Accepts Properties
  - Stateless
- Output: React Element (JSX/TSX)
  - Just like render function in class

```
const HelloWorldHooks =
  (props: IHelloWorldHooksProps): React.ReactElement => {
  return (
    <section className={`${styles.helloWorldHooks} ${hasTeamsContext ? styles
      <div className={styles.welcome}>…
      </div>
      <div>…
      </div>
    </section>
  );
};

export default HelloWorldHooks;
```

WHAT ARE HOOKS?

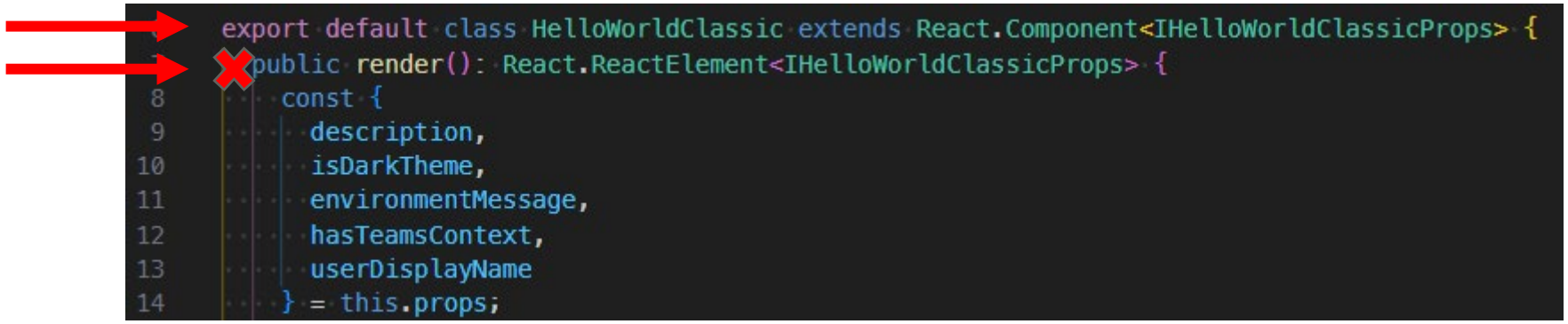
- Primarily used to manage state
- So much more!

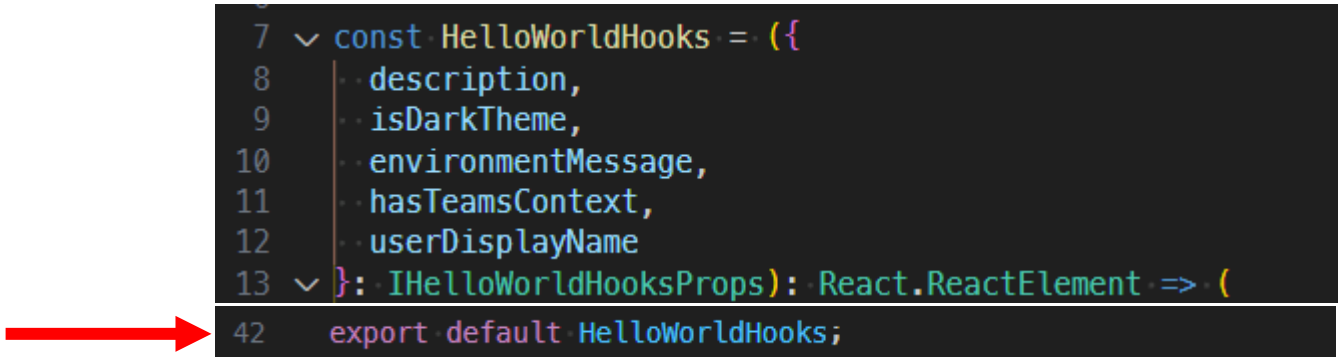
```
const LifecycleHooks: React.FC<ILifecycleHooksProps> = (props) => {
  const [events, setEvents] = React.useState<string[]>(['[lifecycleHooks] co
  const [count, setCount] = React.useState<number>(0);
  const [counting, setCounting] = React.useState<boolean>(false);
  const timerRef = React.useRef<number | undefined>(undefined);
  const didMountRef = React.useRef(false);

  //#region other functions…

  // Simulate componentDidMount/componentWillUnmount
  React.useEffect(() => {…
  }, []);
```

CONVERSION TO FUNCTION COMPONENTS

- Best done right after new project creation
- No changes in the base webpart

- 2 lines + cleanup
  - declaration
  - remove render
  - add export statement





GAMECHANGERS: useState() & useEffect()

useState()

- replaces this.state, granular state
- no lifecycle methods (useEffects)
- useState declared as a 2 item array
  - first item is variable to hold value
  - second item is function to change the variable value
  - initialization is possible in declaration

```
const [events, setEvents] = React.useState<string[]>(['[lifecycle
const [count, setCount] = React.useState<number>(0);
const [counting, setCounting] = React.useState<boolean>(false);
```

```
    setCount(0);
    setCounting(false);
```

```
  if (count === 10) {
```

```
{ counting && <div >Counting...</div> }
```

useEffect()

- function that is called after component renders
- uses 2 parameters
  - 1st parameter is callback function that runs when component renders or updates
  - 2nd parameter is dependency array, which determines when useEffect runs
    - no parameter: always runs
    - []: only runs on first render
    - [value]: runs when value changes
- return function can be used to clean up

```
React.useEffect(() => {
  console.log("Effect runs on every render!");
}); // No dependency array means it runs every render

React.useEffect(() => {
  console.log("Component mounted!");
}, []); // Empty array means it runs only once

React.useEffect(() => {
  console.log(`Count changed to ${count}`);
}, [count]); // Runs only when `count` updates
```

DEMO TIME!

Just warming up!

useCallback

Preventing unnecessary re-renders in child components.

Particularly useful in cases where you pass functions as props to child components or use them inside event handlers, preventing unnecessary re-renders and improving performance.

Takes two arguments:

- **A function** – The callback you want to memoize.
- **A dependency array** – When dependencies change, the function will be re-created.

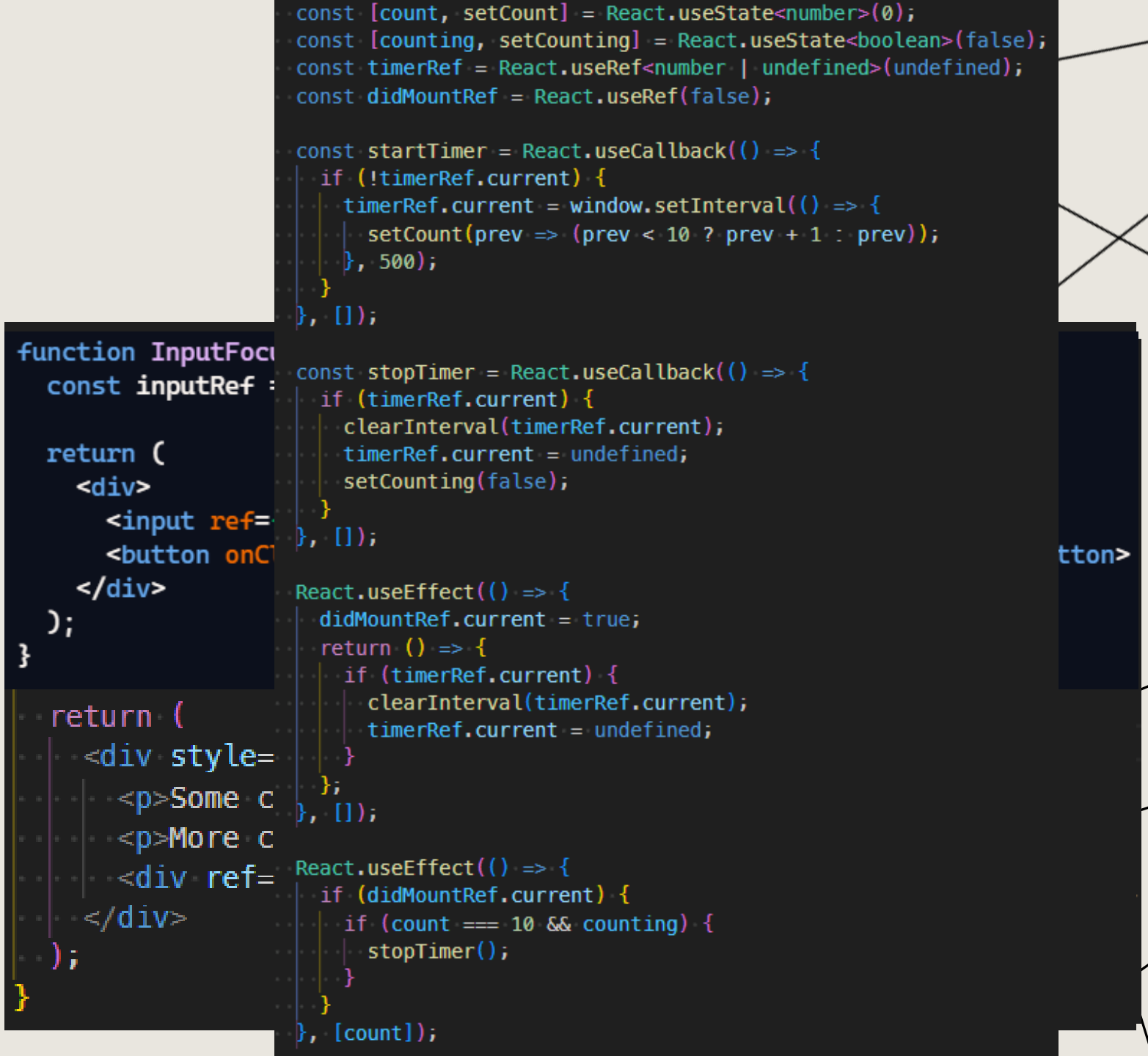
```
function Button({ onClick }) {
  console.log("Button rendered");
  return <button onClick={onClick}>Click me</button>;
}

function ParentComponent() {
  const [count, setCount] = useState(0);

  const increment = useCallback(() => {
    setCount(prevCount => prevCount + 1);
  }, []); // Memoized function

  return (
    <div>
      <p>Count: {count}</p>
      <Button onClick={increment} />
    </div>
  );
}
```

useRef

creates a mutable reference to store values without causing re-renders.

It's commonly used for:

- Accessing & modifying DOM elements (e.g., focusing an input field).
- Persisting values across renders (e.g., tracking previous state or storing intervals).
- Avoiding unnecessary state updates (since useRef doesn't trigger re-renders).



useMemo()

Optimizing performance by memorizing expensive calculations, preventing them from being re-calculated on every render

Takes two arguments:

1. A function – Returns the computed value.
2. A dependency array – The value is only recomputed when dependencies change.

```jsx
function SortedList({ items }) {
  const sortedItems = useMemo(() => {
    console.log("Sorting...");
    return [...items].sort();
  }, [items]);

  return <ul>{sortedItems.map((item, index) => <li key={index}>{item}</li>)}</ul>;
}

function App() {
  const [items, setItems] = useState(["banana", "apple", "cherry"]);

  return (
    <div>
      <SortedList items={items} />
      <button onClick={() => setItems([...items].reverse())}>Reverse</button>
    </div>
  );
}
```

```jsx
function ComputeFactorial({ number }) {
  const factorial = useMemo(() => {
    console.log("Computing factorial...");
    let result = 1;
    for (let i = 2; i <= number; i++) {
      result *= i;
    }
    return result;
  }, [number]);

  return <p>Factorial of {number}: {factorial}</p>;
}
```

```jsx
function Example() {
  const [count, setCount] = useState(0);
  const [query, setQuery] = useState("");
  const items = ["apple", "banana", "cherry", "date", "elderberry"];
  const filteredItems = items.filter(item => item.includes(query));

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} placeholder="Search..." />
      <ul>{filteredItems.map(item => <li key={item}>{item}</li>)}</ul>
      <button onClick={() => setCount(count + 1)}>Count: {count}</button>
    </div>
  );
}
```

useContext()

Allows components to access global state without having to pass props manually at every level.

It's useful for managing theme settings, authentication status, user preferences, or any shared state across multiple components.

How It Works

1. Create a Context – Define shared state using createContext().
2. Provide a Context Value – Use Context.Provider to pass data down.
3. Consume Context in Components – Use useContext() to access the shared state.

Custom Hooks

Encapsulates reusable logic into functions, making code cleaner and more modular.

They’re just JavaScript functions that use built-in hooks like useState(), useEffect(), or useContext() to provide specific functionality.

Why?

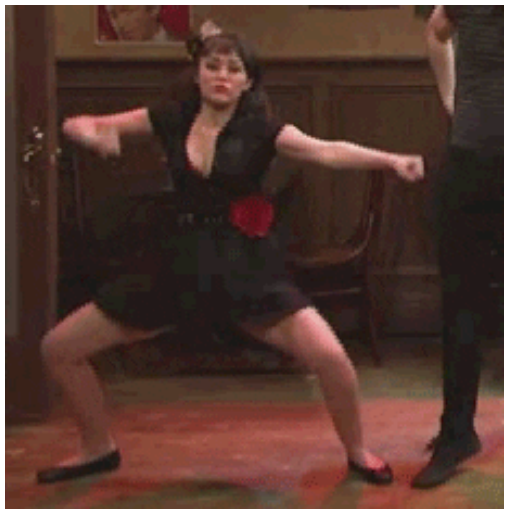
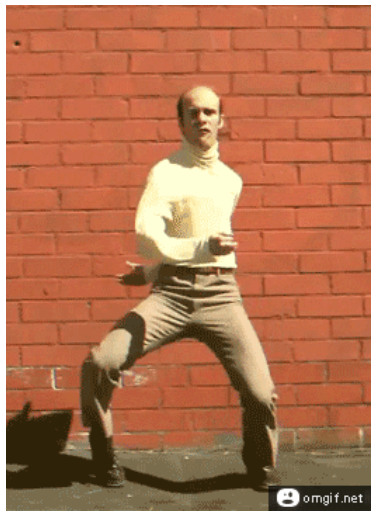
- Avoid repeated logic across multiple components
- Improve code readability and organization
- Encapsulate side effects cleanly

```jsx
import { useState, useEffect } from "react";

function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    const handleResize = () => {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    };

    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return size;
}

function Component() {
  const { width, height } = useWindowSize();
  return <p>Window size: {width}x{height}</p>;
}
```

DEMO TIME!

PUT ON YOUR DANCING SHOES AND LET'S HAVE SOME FUN!



‡–} a„v a™ v»–7 v

❤️

“1va»v „a)v)LO» »v»»O–8 O8

https://donkirkham.com
@DonKirkham
/in/DonKirkham
DonKirkham